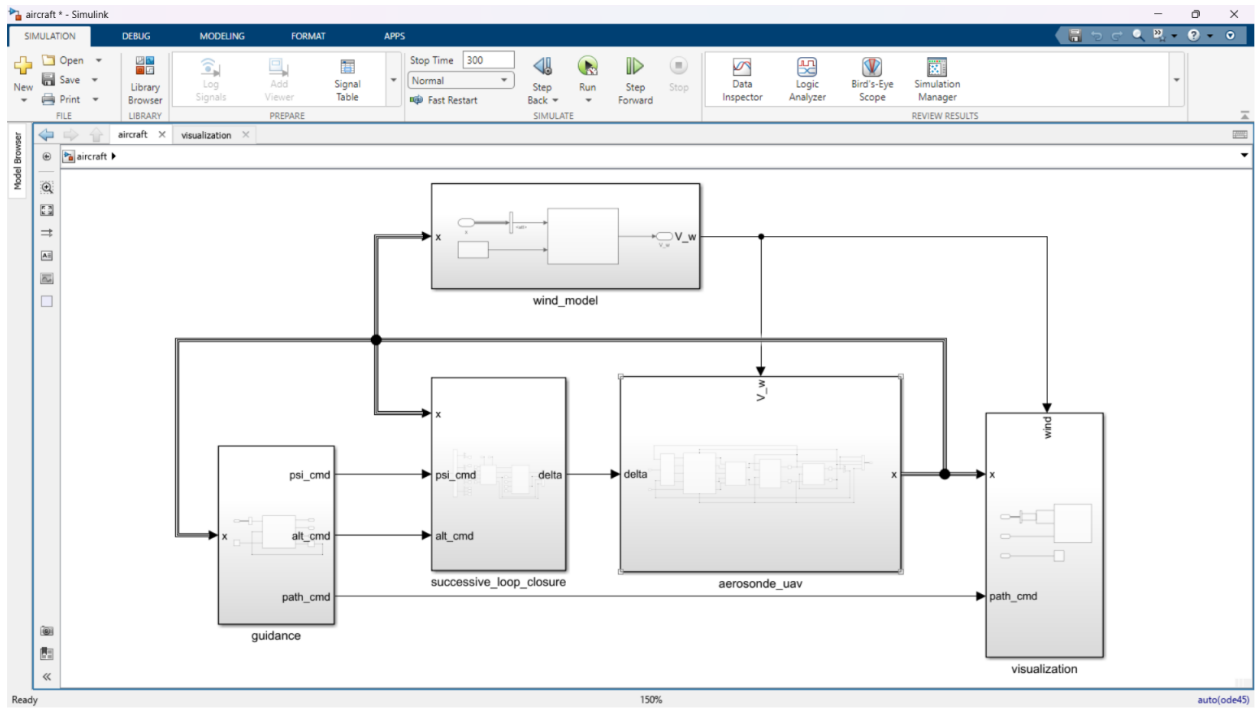
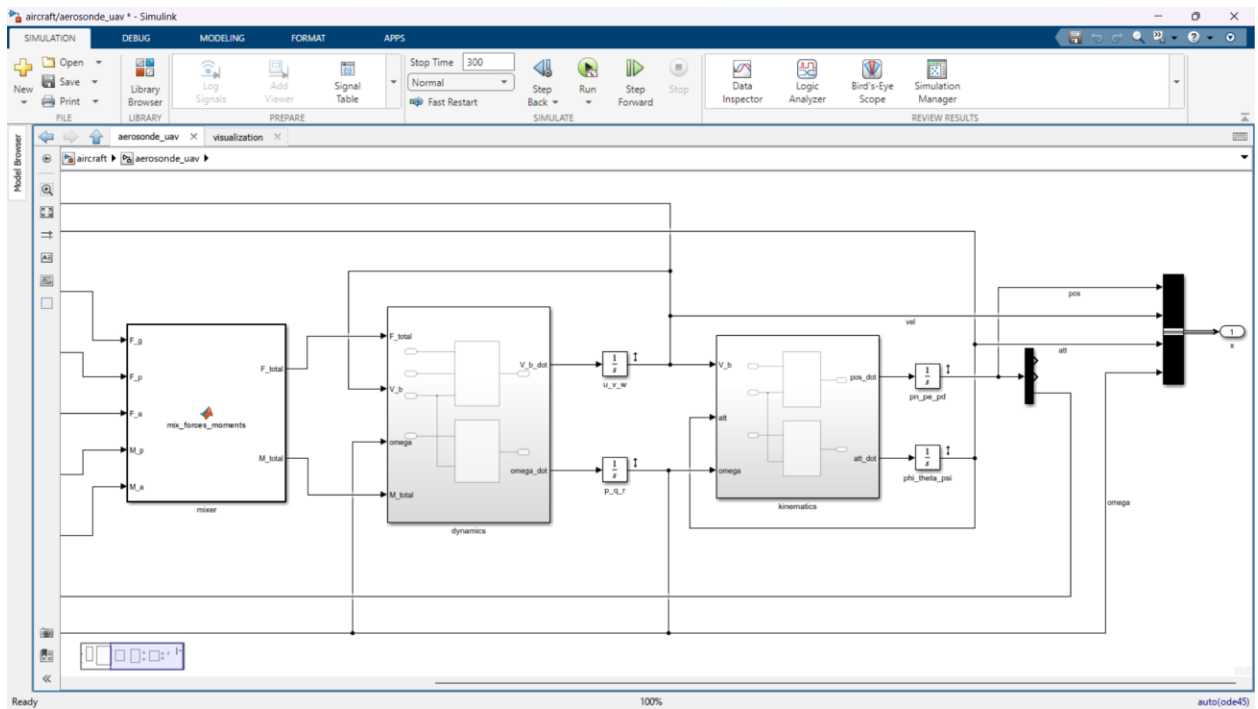
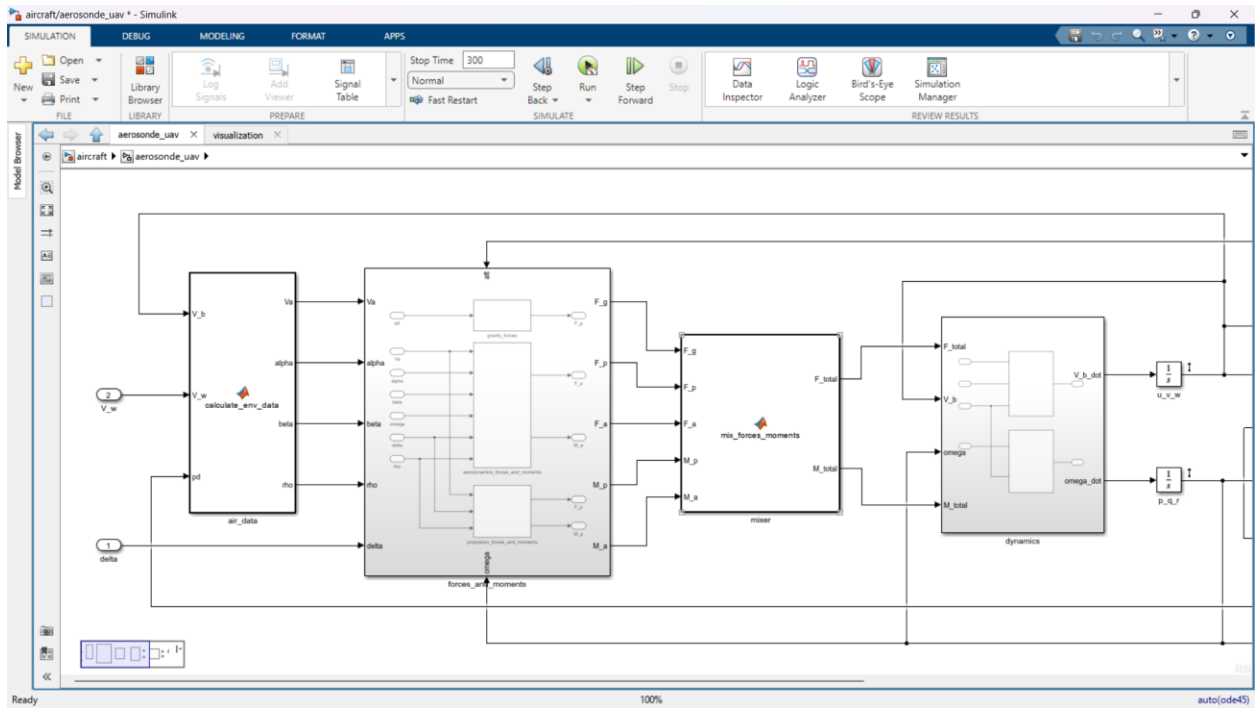


GENERAL SYSTEM



aerosonde_uav (Plant SubSystem)



air_data

```
function [Va, alpha, beta, rho] = calculate_env_data(V_b, V_w, pd, P)
% inputs:
% V_b: [3x1] body-frame ground velocity [u; v; w] (m/s)
% V_w: [3x1] body-frame wind velocity [uw; vw; ww] (m/s)
% pd: [1x1] down position (negative of altitude) (m)
```

```

% outputs:
% Va: [1x1] airspeed magnitude (m/s)
% alpha: [1x1] angle of attack (rad)
% beta: [1x1] sideslip angle (rad)

% current height
h = -pd;

% temperature and pressure at altitude h
T = P.T0 + P.L0 * h;
p = P.p0 * (P.T0 / T)^((P.g * P.Molar) / (P.R * P.L0));

% dynamic density using ideal gas law
rho = (p * P.Molar) / (P.R * T);

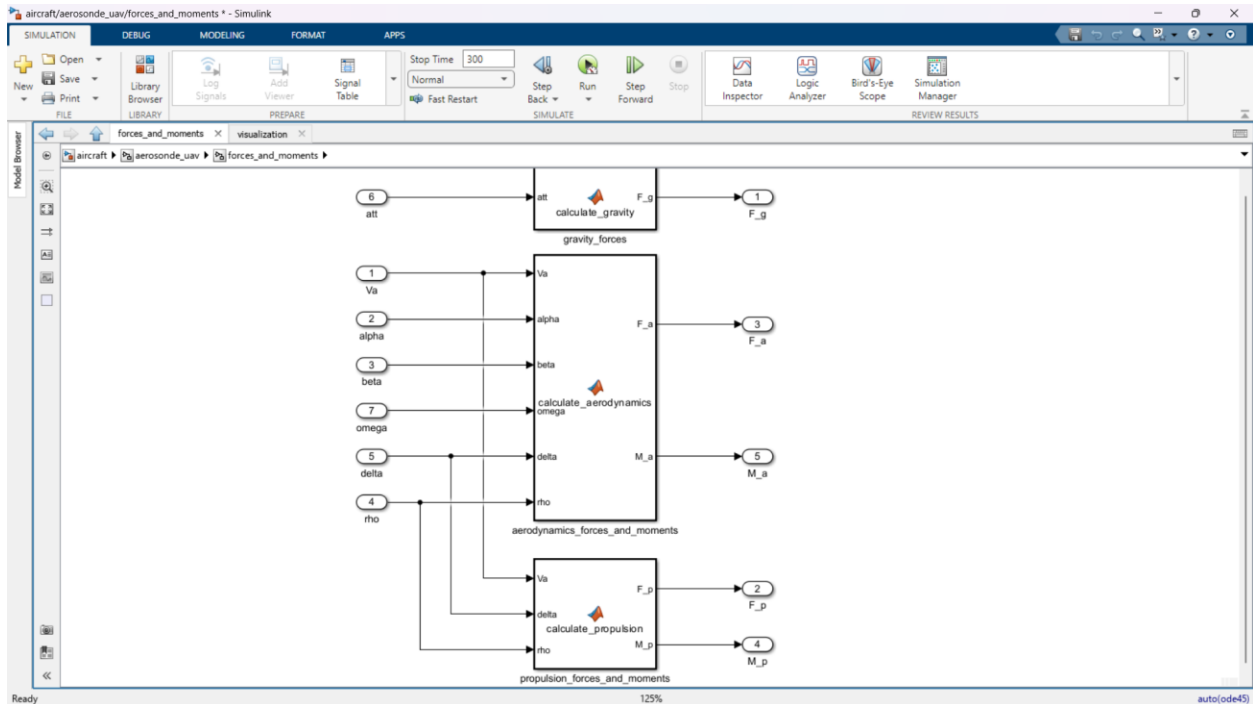
% airspeed vector
V_r = V_b - V_w;

% airspeed magnitude
Va = norm(V_r);

% angle of attack and sideslip
if Va == 0 % prevent to divide by zero
    alpha = 0;
    beta = 0;
else
    alpha = atan2(V_r(3), V_r(1));
    beta = asin(V_r(2) / Va);
end
end
end

```

forces_and_moments



```

function F_g = calculate_gravity(att, P)
% inputs:
% att: [3x1] euler angles [phi; theta; psi] (rad)
% mass: [1x1] uav mass (kg)
% g: [1x1] gravitational constant (m/s^2)
% outputs:
% F_g: [3x1] gravity forces [fx; fy; fz] (n)

phi = att(1);

```

```

theta = att(2);

% project gravity to body frame
F_g = P.mass * P.g * [-sin(theta);
                      cos(theta) * sin(phi);
                      cos(theta) * cos(phi)];
end

function [F_a, M_a] = calculate_aerodynamics(Va, alpha, beta, ...
                                             omega, delta, rho, P)

% inputs:
% Va: [1x1] airspeed (m/s)
% alpha: [1x1] angle of attack (rad)
% beta: [1x1] sideslip angle (rad)
% omega: [3x1] body angular rates [p; q; r] (rad/s)
% delta: [4x1] control signals [da; de; dr; dt]
% rho: [1x1] air density (kg/m^3)
% P: [1x1 struct] aerodynamic parameters
% outputs:
% F_a: [3x1] aerodynamic forces [fax; fay; faz] (n)
% M_a: [3x1] aerodynamic moments [max; may; maz] (n-m)

p = omega(1);
q = omega(2);
r = omega(3);

da = delta(1);
de = delta(2);
dr = delta(3);

% prevent division by zero at rest
if Va == 0
    F_a = [0; 0; 0];
    M_a = [0; 0; 0];
    return;
end

% sigmoid blending function for non-linear stall
sig_num = 1 + exp(-P.M * (alpha - P.alpha0)) + ...
    exp(P.M * (alpha + P.alpha0));
sig_den = (1 + exp(-P.M * (alpha - P.alpha0))) * ...
    (1 + exp(P.M * (alpha + P.alpha0)));
sigma_alpha = sig_num / sig_den;

% non-linear Lift coefficient
CL_linear = P.C_L0 + P.C_Lalpha * alpha;
CL_flat = 2 * sign(alpha) * sin(alpha)^2 * cos(alpha);
CL_alpha = (1 - sigma_alpha) * CL_linear + sigma_alpha * CL_flat;

% non-linear drag coefficient (quadratic)
AR = (P.b^2) / P.S;
CD_alpha = P.C_D0 + (CL_linear^2) / (pi * P.e * AR);

% Longitudinal aerodynamic coefficients
cx = -CD_alpha * cos(alpha) + CL_alpha * sin(alpha);
cx_q = -P.C_Dq * cos(alpha) + P.C_Lq * sin(alpha);
cx_de = -P.C_Dde * cos(alpha) + P.C_Lde * sin(alpha);

cz = -CD_alpha * sin(alpha) - CL_alpha * cos(alpha);
cz_q = -P.C_Dq * sin(alpha) - P.C_Lq * cos(alpha);
cz_de = -P.C_Dde * sin(alpha) - P.C_Lde * cos(alpha);

% dynamic pressure terms
q_bar = 0.5 * rho * Va^2 * P.S;
c_2v = (P.c * q) / (2 * Va);
b_2v_p = (P.b * p) / (2 * Va);
b_2v_r = (P.b * r) / (2 * Va);

% compute forces
fax = q_bar * (cx + cx_q * c_2v + cx_de * de);
fay = q_bar * (P.C_Y0 + P.C_Ybeta * beta + P.C_Yp * b_2v_p + ...
    P.C_Yr * b_2v_r + P.C_Yda * da + P.C_Ydr * dr);
faz = q_bar * (cz + cz_q * c_2v + cz_de * de);

% compute moments
max = q_bar * P.b * (P.C_Mx0 + P.C_Mxbeta * beta + ...
    P.C_Mxp * b_2v_p + P.C_Mxr * b_2v_r + ...
    P.C_Mxda * da + P.C_Mxdr * dr);
may = q_bar * P.c * (P.C_My0 + P.C_Myalpha * alpha + ...

```

```

        P.C_Myq * c_2v + P.C_Myde * de);
maz = q_bar * P.b * (P.C_Mz0 + P.C_Mzbeta * beta + ...
P.C_Mzp * b_2v_p + P.C_Mzr * b_2v_r + ...
P.C_Mzda * da + P.C_Mzdr * dr);

F_a = [fax; fay; faz];
M_a = [max; may; maz];
end
function [F_p, M_p] = calculate_propulsion(Va, delta, rho, P)
% calculates propulsion forces and moments[cite: 1]
% inputs:
% Va: [1x1] airspeed (m/s)
% delta:[4x1] control signals [da; de; dr; dt]
% rho: [1x1] air density (kg/m^3)
% P: [1x1 struct] physical parameters
% outputs:
% F_p: [3x1] propulsive force [fx; fy; fz] (n)
% M_p: [3x1] propulsive moment [mx; my; mz] (n-m)

dt = delta(4);
if dt == 0
    F_p = [0; 0; 0];
    M_p = [0; 0; 0];
    return;
end

v_in = P.V_max * dt;

a = (rho * P.D_prop^5) / ((2 * pi)^2 * P.C_Q0;
b = (rho * P.D_prop^4) / (2 * pi) * P.C_Q1 * Va + ...
(P.k_Q * P.k_V) / P.R_motor;
c = rho * P.D_prop^3 * P.C_Q2 * Va^2 - ...
(P.k_Q / P.R_motor) * v_in + P.k_Q * P.i0;

omega_p = (-b + sqrt(b^2 - 4 * a * c)) / (2 * a);

t_p = (rho * P.D_prop^4) / ((2 * pi)^2 * omega_p^2 * P.C_T0 + ...
(rho * P.D_prop^3) / (2 * pi) * omega_p * Va * P.C_T1 + ...
rho * P.D_prop^2 * Va^2 * P.C_T2;

q_p = (rho * P.D_prop^5) / ((2 * pi)^2 * omega_p^2 * P.C_Q0 + ...
(rho * P.D_prop^4) / (2 * pi) * omega_p * Va * P.C_Q1 + ...
rho * P.D_prop^3 * Va^2 * P.C_Q2;

F_p = [t_p; 0; 0];
M_p = [-q_p; 0; 0];
end

```

mixer

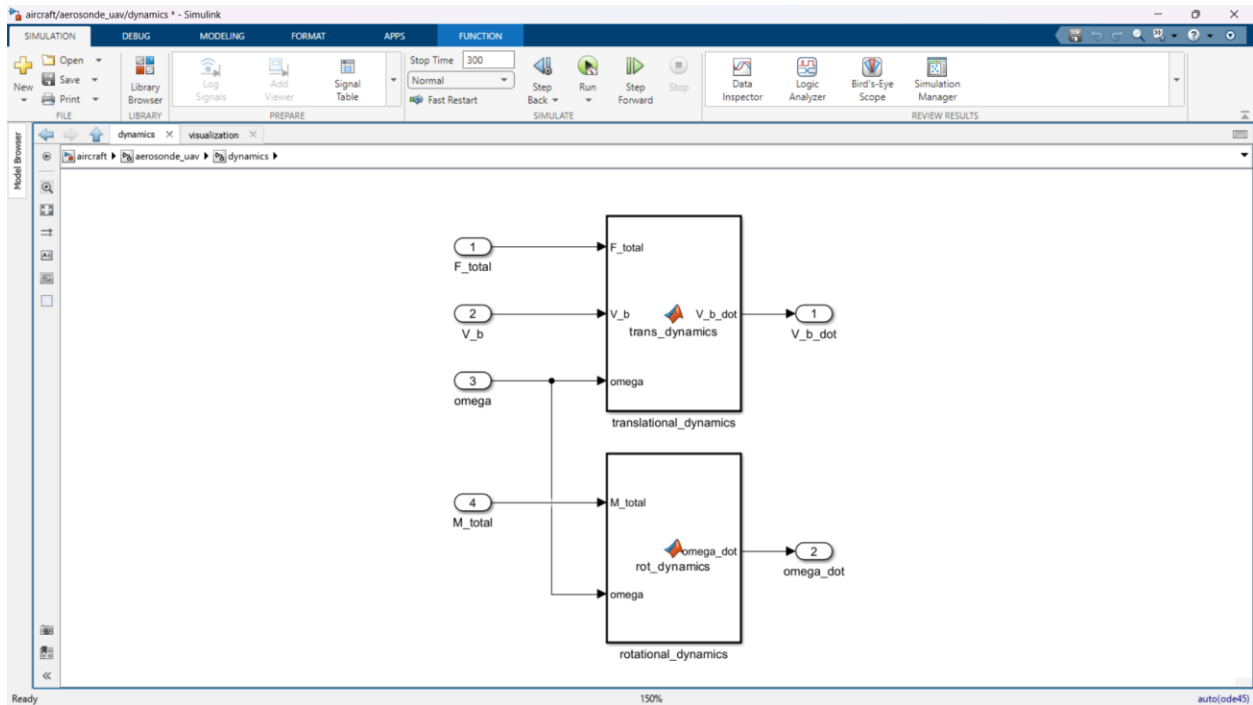
```

function [F_total, M_total] = mix_forces_moments(F_g, F_p, F_a, M_p, M_a)
% sums all forces and moments[cite: 1]
% inputs:
% F_g: [3x1] gravity force (n)
% F_p: [3x1] propulsive force (n)
% F_a: [3x1] aerodynamic force (n)
% M_p: [3x1] propulsive moment (n-m)
% M_a: [3x1] aerodynamic moment (n-m)
% outputs:
% F_total: [3x1] total body frame force (n)
% M_total: [3x1] total body frame moment (n-m)

F_total = F_g + F_p + F_a;
M_total = M_p + M_a;
end

```

dynamics



```
function V_b_dot = trans_dynamics(F_total, V_b, omega, P)
% inputs:
% F_total: [3x1] total forces [fx; fy; fz] (n)
% V_b: [3x1] body velocities [u; v; w] (m/s)
% omega: [3x1] angular rates [p; q; r] (rad/s)
% P: [1x1 struct] physical parameters
% outputs:
% V_b_dot: [3x1] velocity derivatives [u_dot; v_dot; w_dot] (m/s^2)

u = V_b(1); v = V_b(2); w = V_b(3);
p = omega(1); q = omega(2); r = omega(3);
fx = F_total(1);
fy = F_total(2);
fz = F_total(3);

u_dot = r*v - q*w + fx / P.mass;
v_dot = p*w - r*u + fy / P.mass;
w_dot = q*u - p*v + fz / P.mass;

V_b_dot = [u_dot; v_dot; w_dot];
end

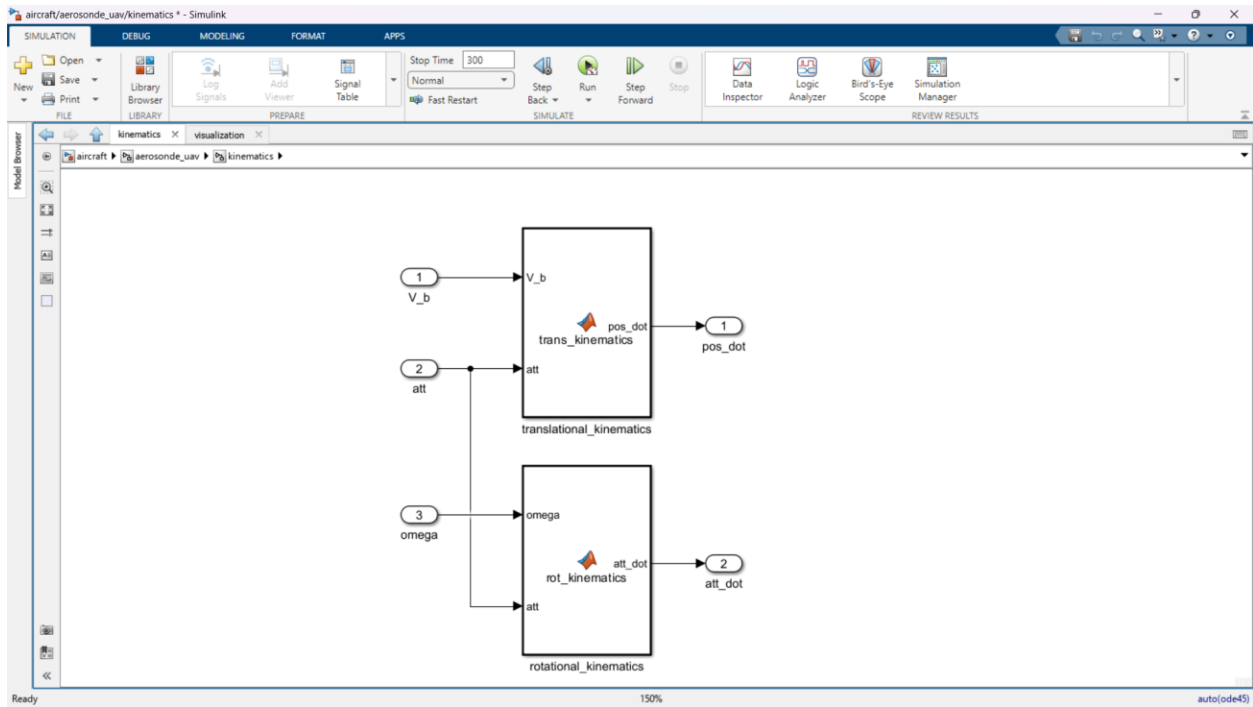
function omega_dot = rot_dynamics(M_total, omega, P)
% inputs:
% M_total: [3x1] total moments [mx; my; mz] (n-m)
% omega: [3x1] angular rates [p; q; r] (rad/s)
% P: [1x1 struct] physical parameters
% outputs:
% omega_dot: [3x1] angular rate derivatives [p_dot; q_dot; r_dot] (rad/s^2)

p = omega(1); q = omega(2); r = omega(3);
mx = M_total(1); my = M_total(2); mz = M_total(3);

p_dot = P.Gamma1*p*q - P.Gamma2*q*r + P.Gamma3*mx + P.Gamma4*mz;
q_dot = P.Gamma5*p*r - P.Gamma6*(p^2 - r^2) + my / P.Jy;
r_dot = P.Gamma7*p*q - P.Gamma1*q*r + P.Gamma4*mx + P.Gamma8*mz;

omega_dot = [p_dot; q_dot; r_dot];
end
```

kinematics



```
function pos_dot = trans_kinematics(V_b, att)
% inputs:
% V_b: [3x1] body velocities [u; v; w] (m/s)
% att: [3x1] euler angles [phi; theta; psi] (rad)
% outputs:
% pos_dot: [3x1] inertial pos derivatives [pn_dot; pe_dot; pd_dot] (m/s)

u = V_b(1); v = V_b(2); w = V_b(3);
phi = att(1); theta = att(2); psi = att(3);

cphi = cos(phi); sphi = sin(phi);
cthe = cos(theta); sthe = sin(theta);
cpsi = cos(psi); spsi = sin(psi);

R_b_v = [cthe*cpsi, sphi*sthe*cpsi - cphi*spsi, cphi*sthe*cpsi + sphi*spsi;
         cthe*spsi, sphi*sthe*spsi + cphi*cpsi, cphi*sthe*spsi - sphi*cpsi;
         -sthe,      sphi*cthe,      cphi*cthe];

pos_dot = R_b_v * [u; v; w];
end

function att_dot = rot_kinematics(omega, att)
% inputs:
% omega: [3x1] angular rates [p; q; r] (rad/s)
% att: [3x1] euler angles [phi; theta; psi] (rad)
% outputs:
% att_dot: [3x1] euler angle derivatives [phi_dot; theta_dot; psi_dot] (rad/s)

p = omega(1); q = omega(2); r = omega(3);
phi = att(1); theta = att(2);

cphi = cos(phi); sphi = sin(phi);
cthe = cos(theta); tthe = tan(theta);

R_rot = [1, sphi*tthe, cphi*tthe;
         0, cphi,      -sphi;
         0, sphi/cthe, cphi/cthe];

att_dot = R_rot * [p; q; r];
end
```

initial condition

 Block Parameters: pn_pe_pd ✕

Integrator

Continuous-time integration of the input signal.

Parameters

External reset:

Initial condition source:

Initial condition:

☐ Limit output

☐ Wrap state

☐ Show saturation port

☐ Show state port

Absolute tolerance:

☐ Ignore limit and reset when linearizing

☒ Enable zero-crossing detection

State Name: (e.g., 'position')

OK

Cancel

Help

Apply

Name:
ShowSaturationPort

 Block Parameters: u_v_w ✕

Integrator

Continuous-time integration of the input signal.

Parameters

External reset:

Initial condition source:

Initial condition:

☐ Limit output

☐ Wrap state

☐ Show saturation port

☐ Show state port

Absolute tolerance:

☐ Ignore limit and reset when linearizing

☒ Enable zero-crossing detection

State Name: (e.g., 'position')

OK

Cancel

Help

Apply

 Block Parameters: phi_theta_psi



Integrator

Continuous-time integration of the input signal.

Parameters

External reset:

none



Initial condition source:

internal



Initial condition:

[0; 0.14622; 0]



☐ Limit output

☐ Wrap state

☐ Show saturation port

☐ Show state port

Absolute tolerance:

auto



☐ Ignore limit and reset when linearizing

☒ Enable zero-crossing detection

State Name: (e.g., 'position')

"

OK

Cancel

Help

Apply

 Block Parameters: p_q_r ✕

Integrator

Continuous-time integration of the input signal.

Parameters

External reset:

Initial condition source:

Initial condition:

☐ Limit output

☐ Wrap state

☐ Show saturation port

☐ Show state port

Absolute tolerance:

☐ Ignore limit and reset when linearizing

☒ Enable zero-crossing detection

State Name: (e.g., 'position')

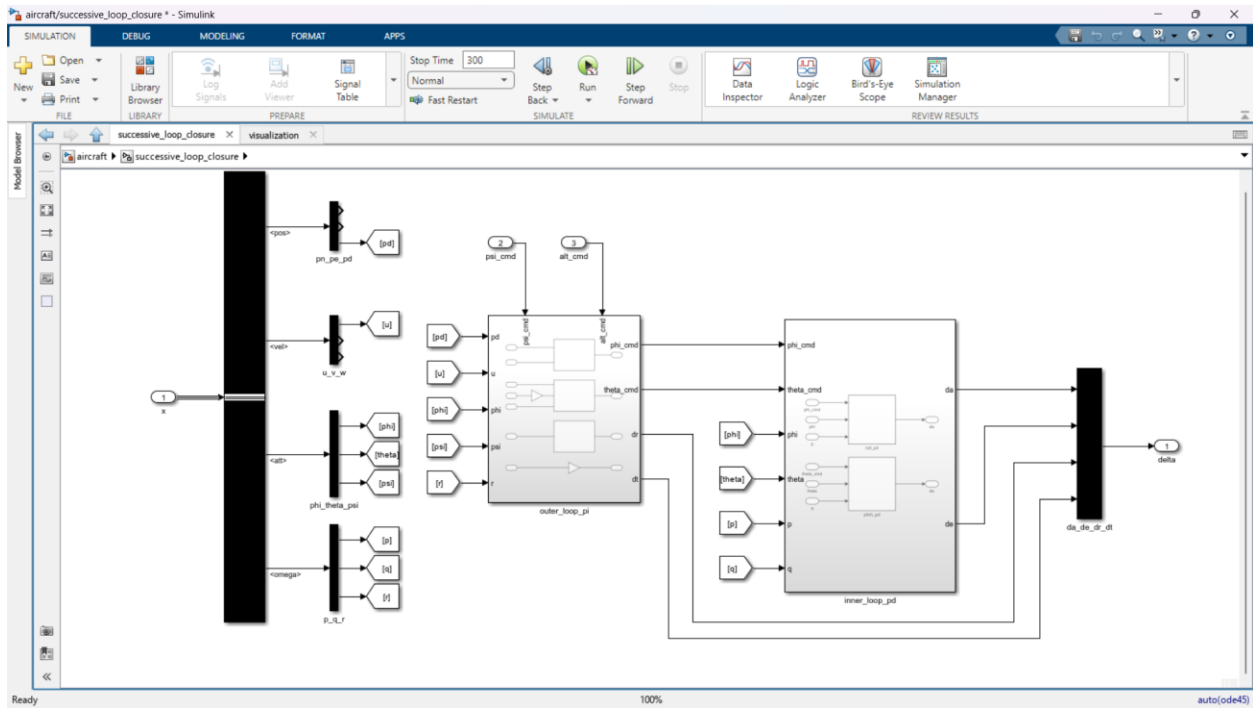
OK

Cancel

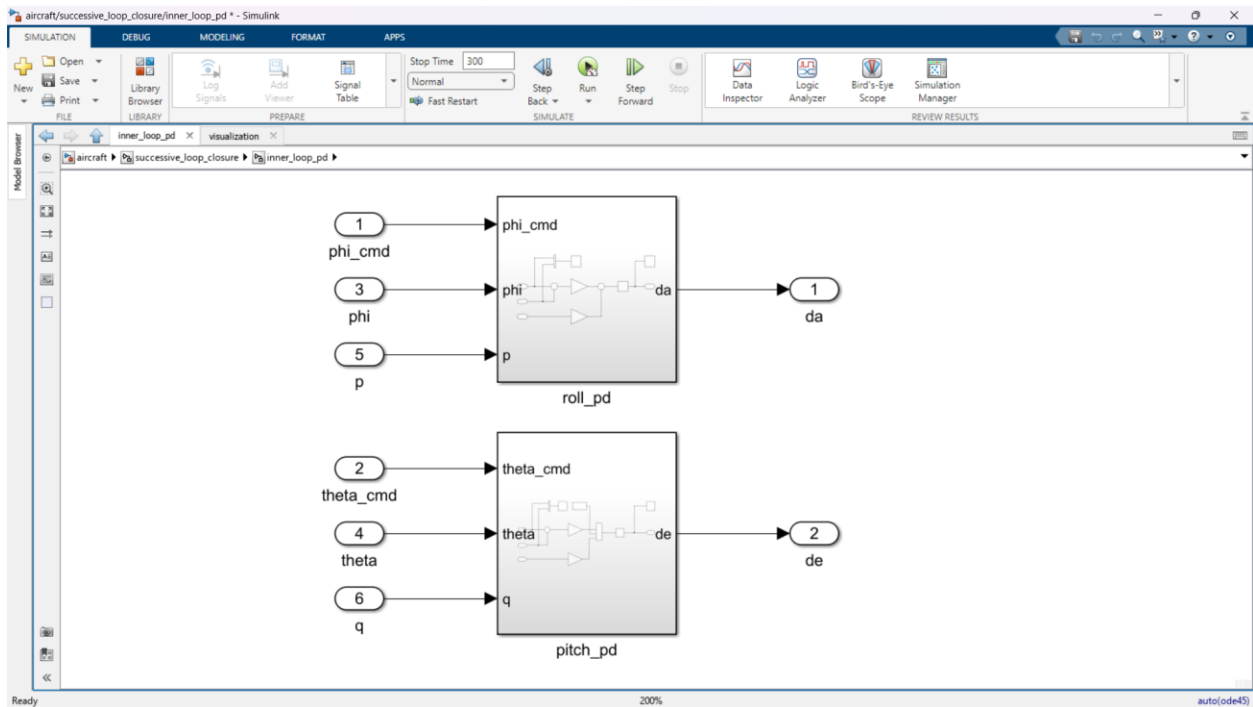
Help

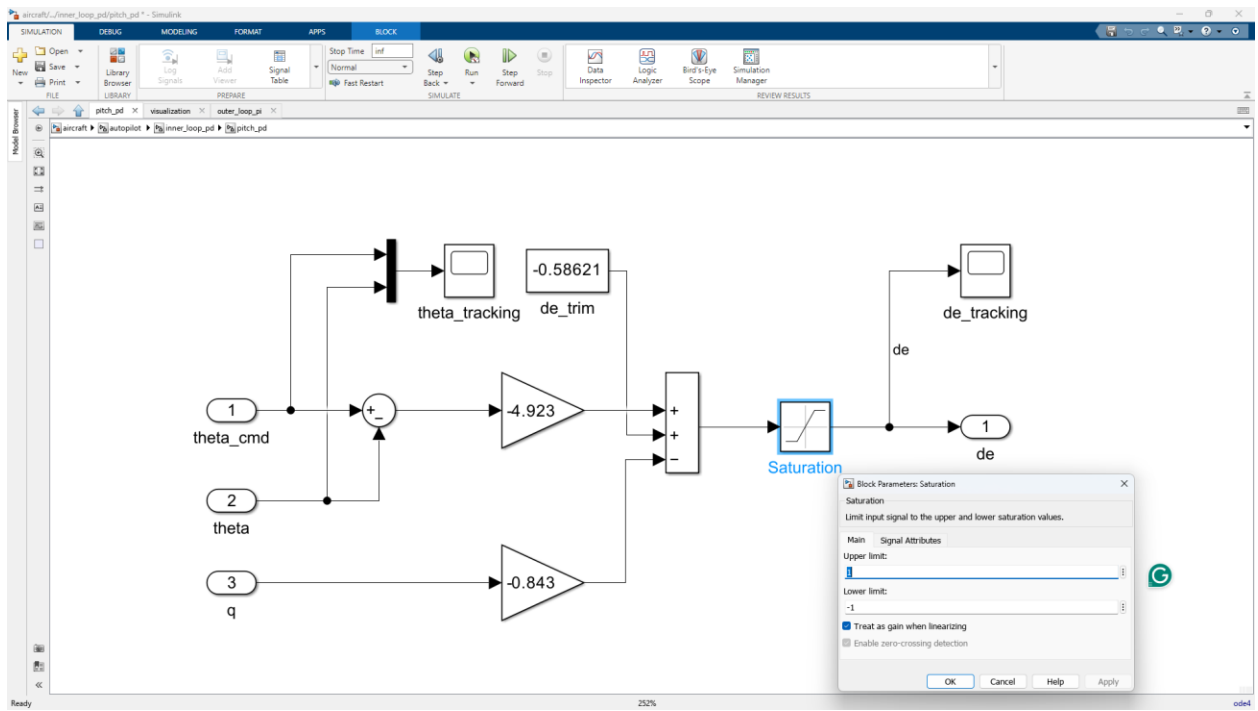
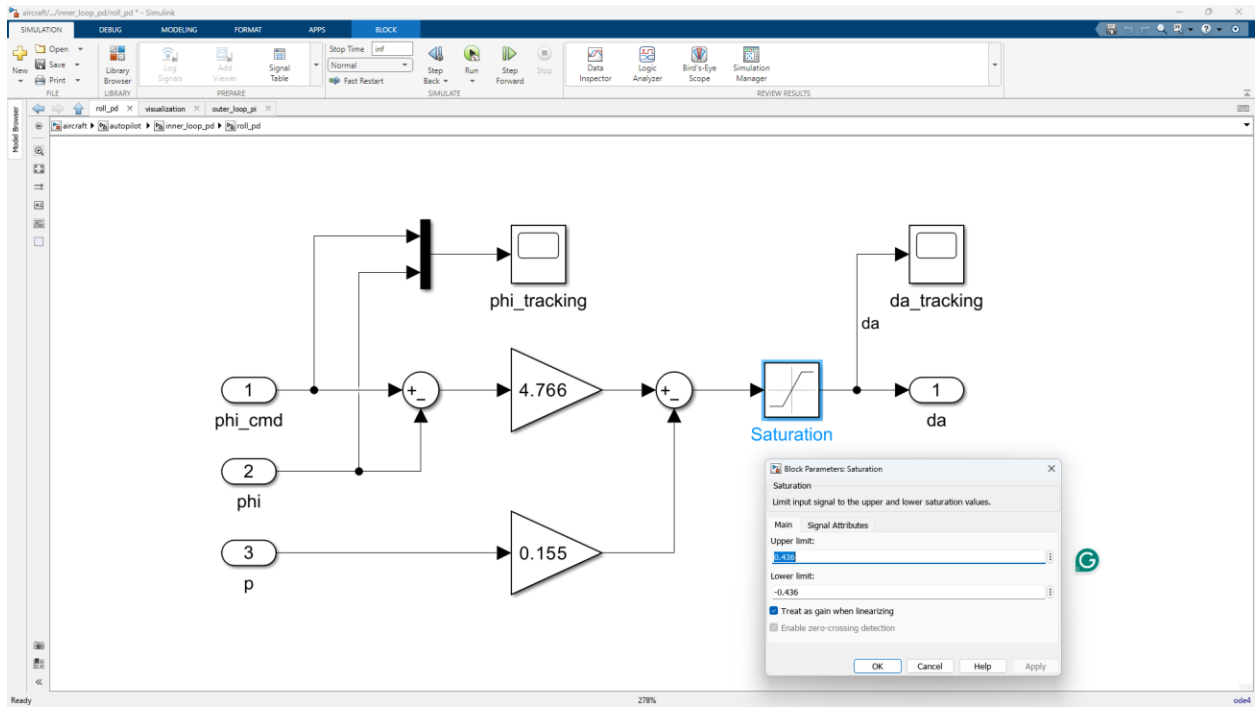
Apply

successive_loop_closure (Controller SubSystem)

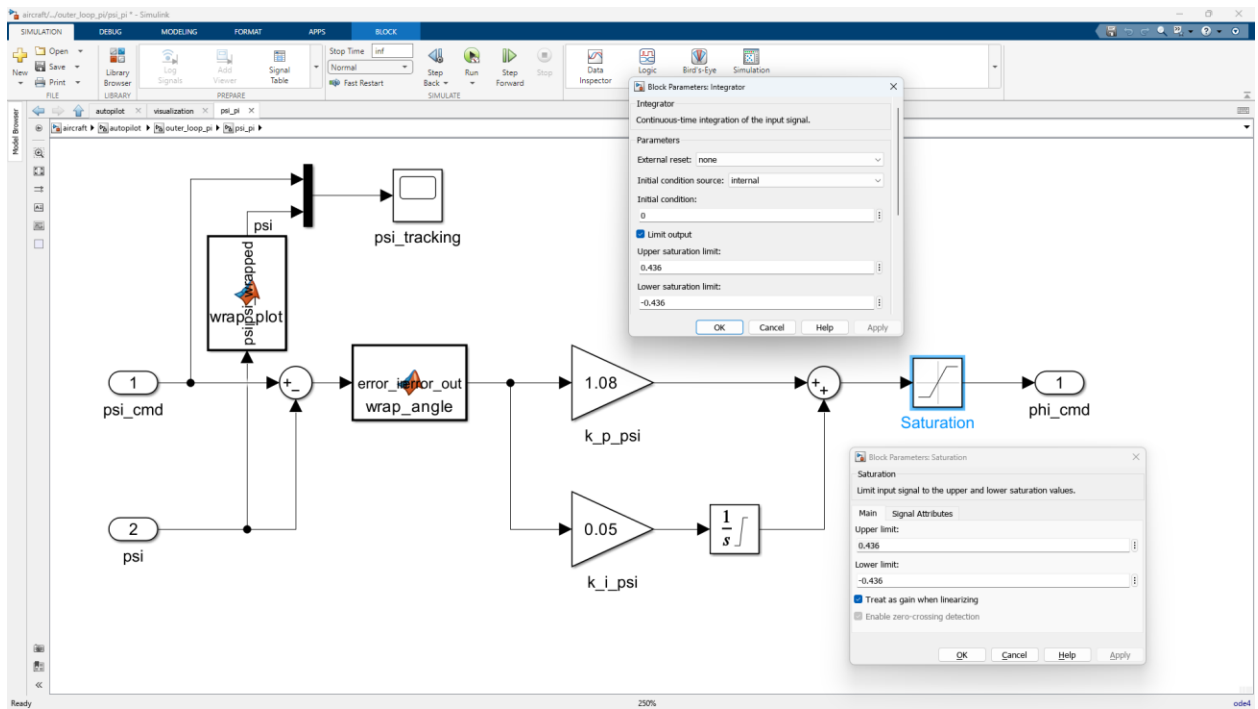
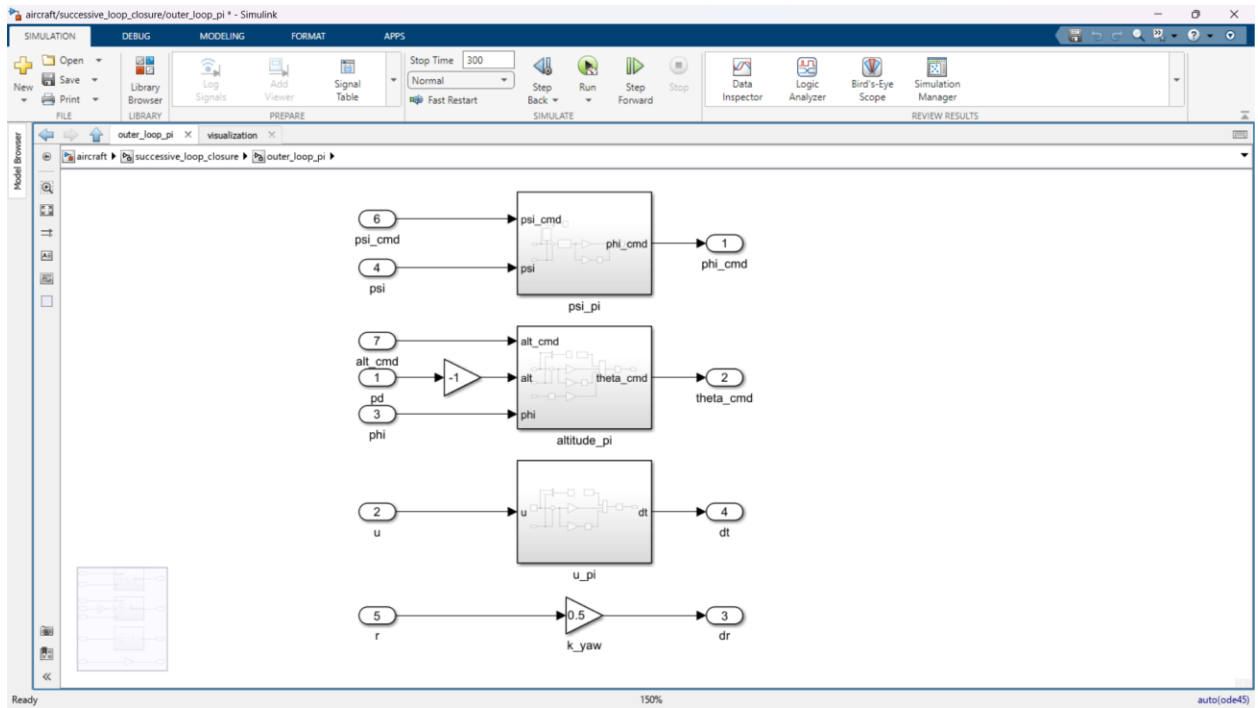


inner_loop_pd



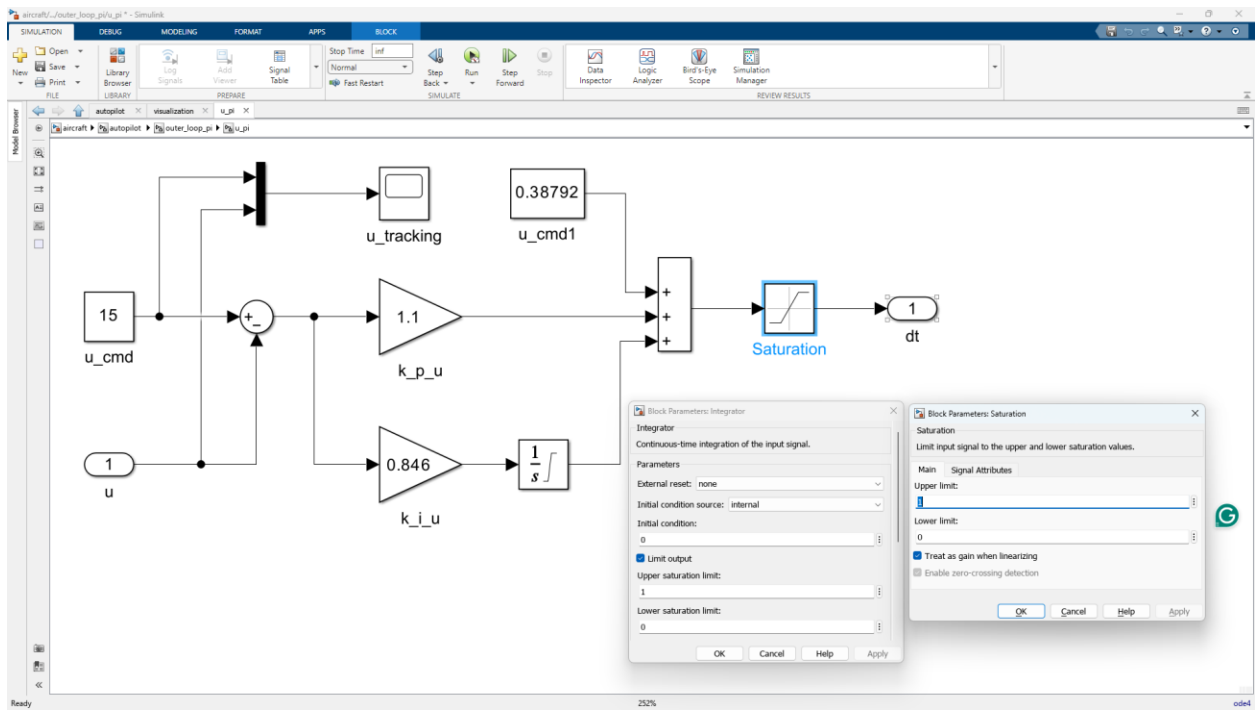
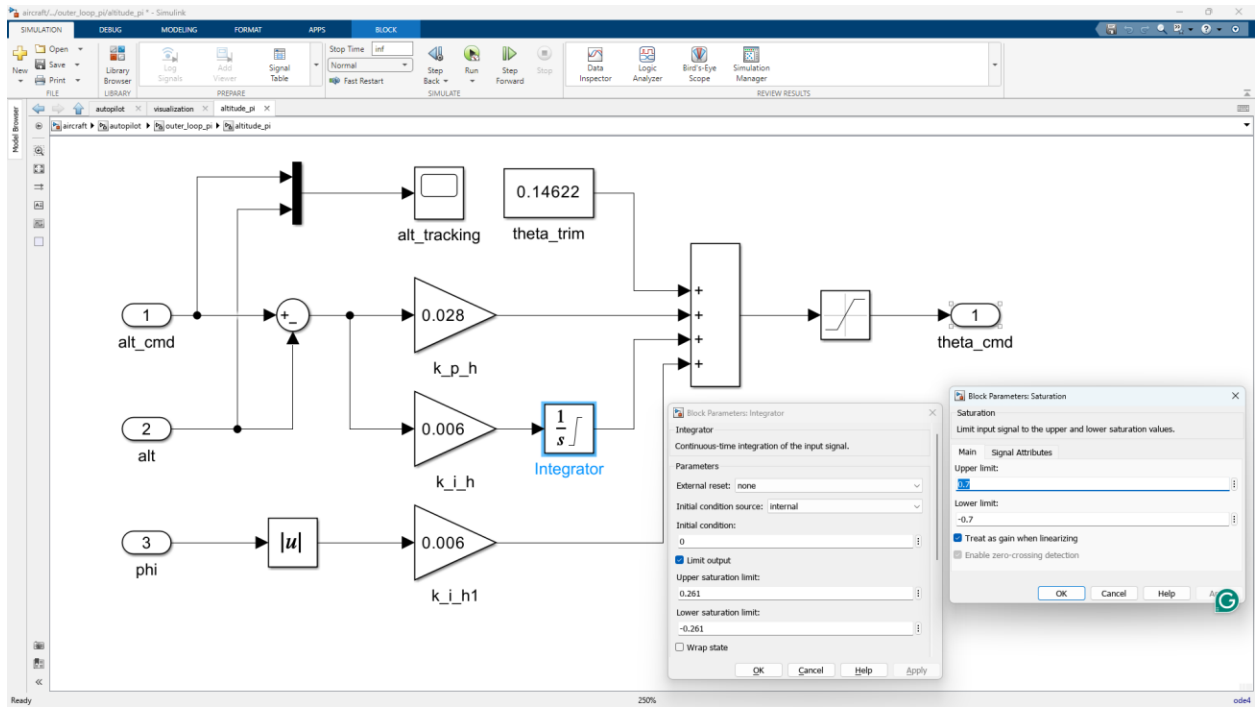


outer_loop_pi

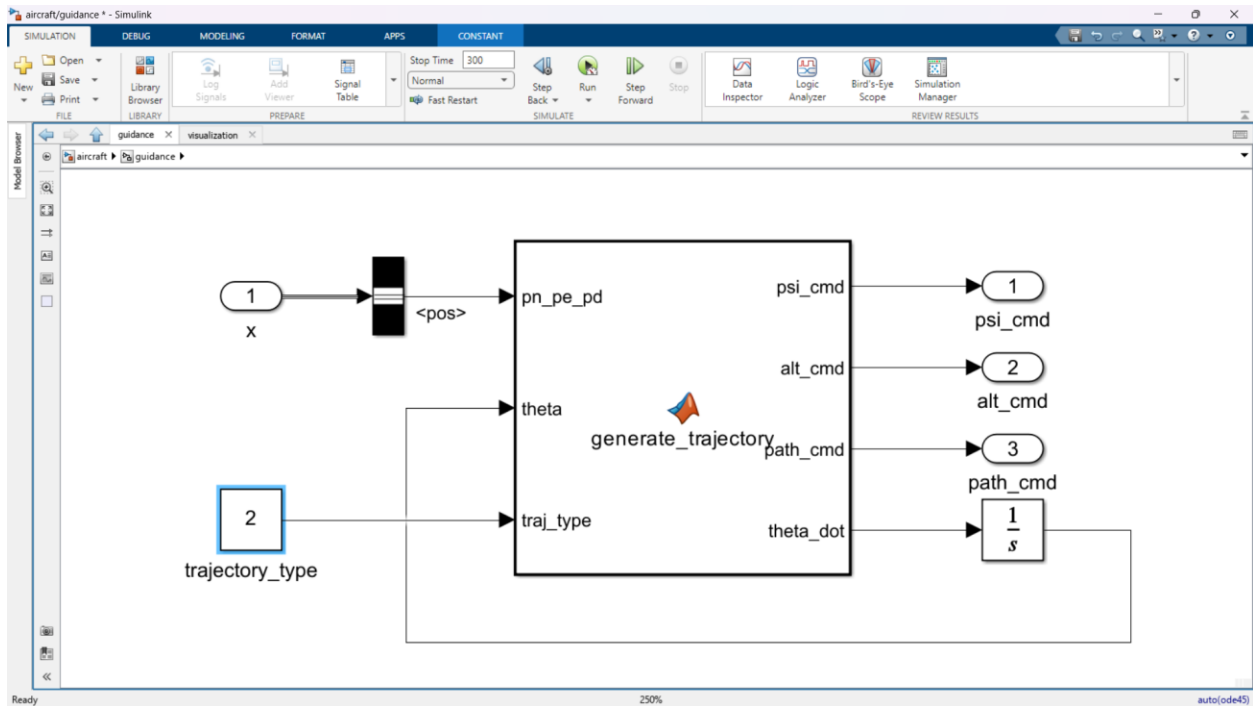


```
function error_out = wrap_angle(error_in)
    % wraps any angle to strictly [-pi, pi]
    error_out = mod(error_in + pi, 2*pi) - pi;
end

function psi_wrapped = wrap_plot(psi)
    psi_wrapped = mod(psi + pi, 2*pi) - pi;
end
```



guidance (Guidance SubSystem)



```
function [psi_cmd, alt_cmd, path_cmd, theta_dot] = generate_trajectory(pn_pe_pd, theta, traj_type)
% Extract current UAV position
pn = pn_pe_pd(1);
pe = pn_pe_pd(2);

A = 300;
V_uav = 15.0;
N_center = 500;
E_center = 500;

if traj_type == 1
    % --- 1. FIGURE-8 (Lemniscate) ---
    % theta represents the CARROT's current position
    N_carrot = N_center + A * sin(theta) * cos(theta);
    E_carrot = E_center + A * sin(theta);
    Alt_carrot = 100;

    % Calculate path gradients at the carrot
    dN_dtheta = A * cos(2 * theta);
    dE_dtheta = A * cos(theta);

else
    % --- 2. HELIX (Climbing Circle) ---
    N_carrot = N_center + A * cos(theta);
    E_carrot = E_center + A * sin(theta);
    Alt_carrot = 100 + (13.0 * theta);

    dN_dtheta = -A * sin(theta);
    dE_dtheta = A * cos(theta);
end

% --- THE VIRTUAL TARGET FEEDBACK LOOP ---
% 1. Calculate actual physical distance between UAV and Carrot
d_actual = sqrt((N_carrot - pn)^2 + (E_carrot - pe)^2);

% 2. Enforce a strict Lookahead Tether (e.g., exactly 40 meters)
L_desired = 40.0;

% 3. Carrot Speed Controller:
% If d_actual < 40, carrot speeds up. If d_actual > 40, it slows down.
k_carrot = 1.5; % Tuning gain for target speed
V_carrot = V_uav + k_carrot * (L_desired - d_actual);
```

```

% --- SATURATION: Prevent supersonic carrot speeds ---
if V_carrot > 25.0
    V_carrot = 25.0; % Max sprint speed
elseif V_carrot < 0.0
    V_carrot = 0.0; % Stop and wait for UAV
end

% 4. Convert linear carrot speed to angular geometric speed
gradient = sqrt(dN_dtheta^2 + dE_dtheta^2);
theta_dot = V_carrot / gradient;

% --- THE GUIDANCE LAW ---
chi_los = atan2(E_carrot - pe, N_carrot - pn);
chi_q = atan2(dE_dtheta, dN_dtheta);

error_los = mod((chi_los - chi_q) + pi, 2*pi) - pi;

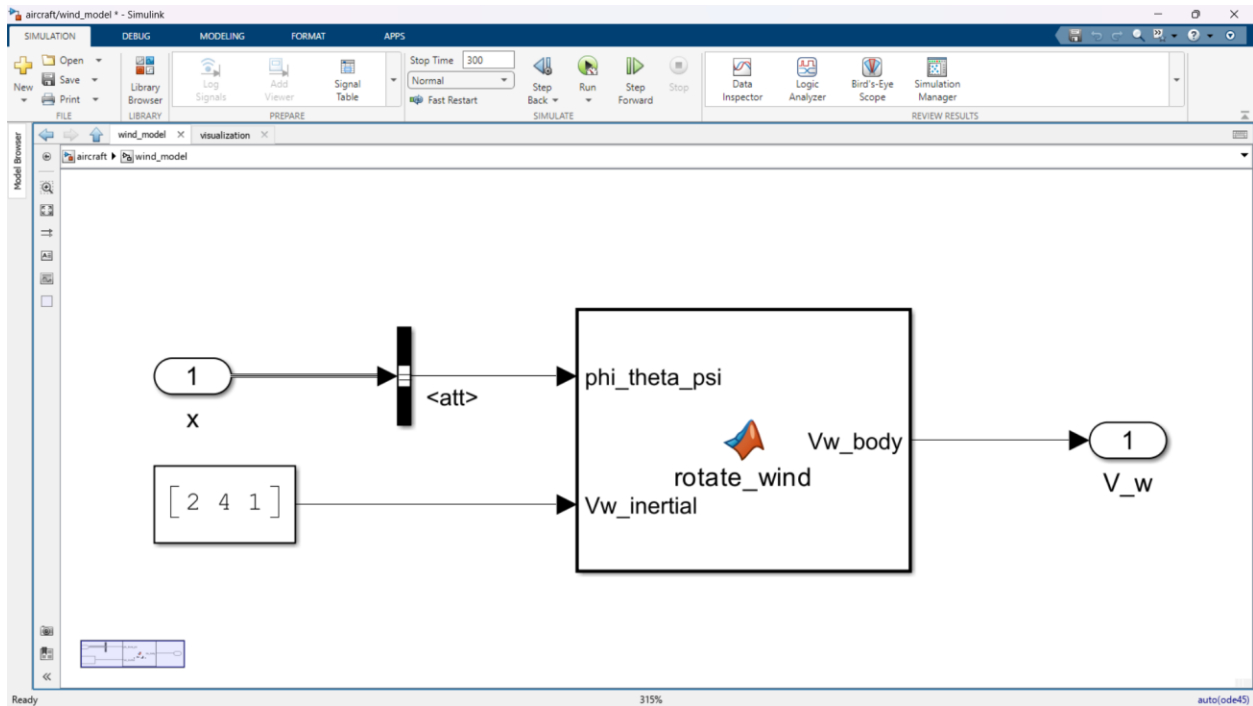
% Heading Command = Path Tangent + Proportional Correction
k_path = 1.2;
psi_cmd = chi_q + (k_path * error_los);

alt_cmd = Alt_carrot;

% Output for 3D visualizer
path_cmd = [N_carrot; E_carrot; Alt_carrot; traj_type];
end

```

wind_model (Disturbance SubSystem)



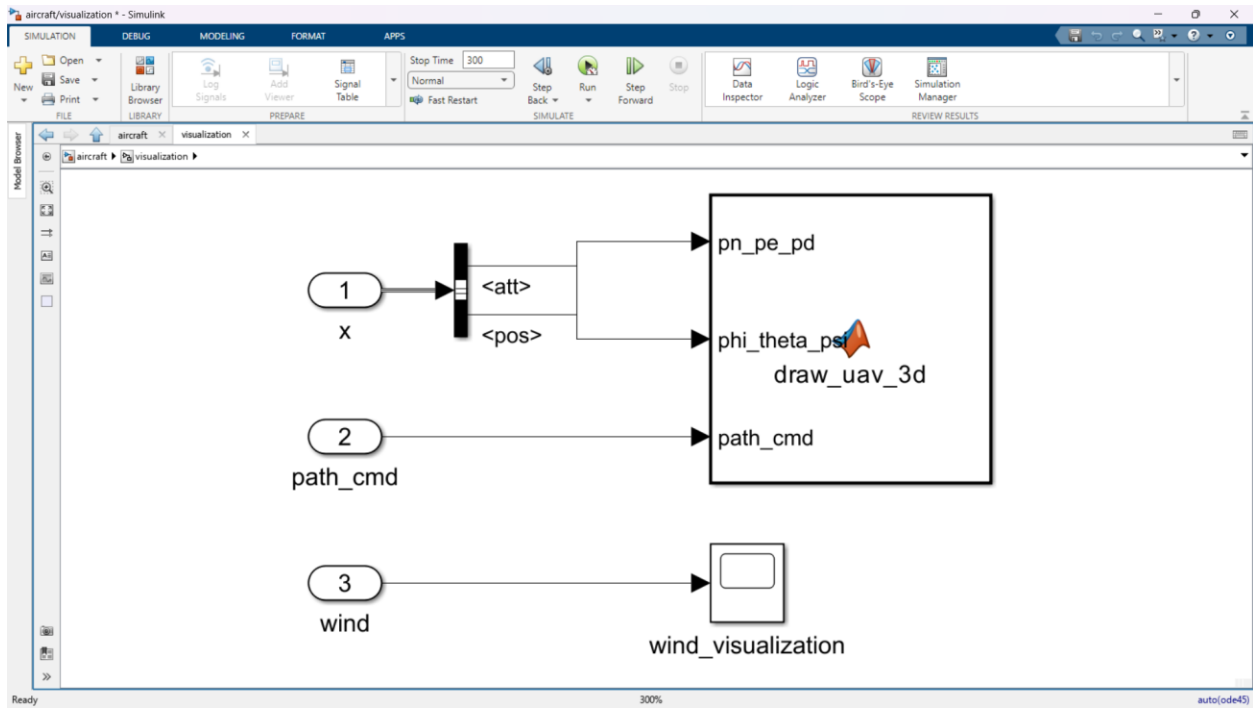
```
function Vw_body = rotate_wind(phi_theta_psi, Vw_inertial)
    phi = phi_theta_psi(1);
    theta = phi_theta_psi(2);
    psi = phi_theta_psi(3);

    % Rotation Matrix from Inertial to Body Frame (R_v^b)
    R_yaw = [cos(psi), sin(psi), 0; -sin(psi), cos(psi), 0; 0, 0, 1];
    R_pitch = [cos(theta), 0, -sin(theta); 0, 1, 0; sin(theta), 0, cos(theta)];
    R_roll = [1, 0, 0; 0, cos(phi), sin(phi); 0, -sin(phi), cos(phi)];

    R_v_to_b = R_roll * R_pitch * R_yaw;

    % Total body frame wind (Ready for gusts to be added later)
    Vw_body = R_v_to_b * Vw_inertial;
end
```

visualization (Visualization SubSystem)



```
function draw_uav_3d(pn_pe_pd, phi_theta_psi, path_cmd)
    coder.extrinsic('update_uav_plot');
    update_uav_plot(pn_pe_pd, phi_theta_psi, path_cmd);
end
```

MATLAB CODE

load_uav_params.m

```
% =====
% AEROSONDE UAV PARAMETERS
% =====

%% --- GRAVITY AND MASS ---
P.g = 9.81; % Gravitational constant (m/s^2)
P.mass = 11.0; % Aircraft mass (kg)

%% --- MOMENTS AND PRODUCTS OF INERTIA ---
% These determine the aircraft's resistance to angular acceleration.
P.Jx = 0.824; % Roll moment of inertia (kg-m^2)
P.Jy = 1.135; % Pitch moment of inertia (kg-m^2)
P.Jz = 1.759; % Yaw moment of inertia (kg-m^2)
P.Jxz = 0.120; % Product of inertia (kg-m^2)

%% --- INERTIA KINEMATIC CONSTANTS ---
% Pre-computed constants used in the rigid-body rotational equations.
Gamma = P.Jx*P.Jz - P.Jxz^2;
P.Gamma1 = P.Jxz*(P.Jx - P.Jy + P.Jz) / Gamma;
P.Gamma2 = (P.Jz*(P.Jz - P.Jy) + P.Jxz^2) / Gamma;
P.Gamma3 = P.Jz / Gamma;
P.Gamma4 = P.Jxz / Gamma;
P.Gamma5 = (P.Jz - P.Jx) / P.Jy;
P.Gamma6 = P.Jxz / P.Jy;
P.Gamma7 = ((P.Jx - P.Jy)*P.Jx + P.Jxz^2) / Gamma;
P.Gamma8 = P.Jx / Gamma;

%% --- AIRCRAFT GEOMETRY ---
P.S = 0.55; % Wing planform area (m^2)
P.b = 2.9; % Wingspan (m)
P.c = 0.19; % Mean aerodynamic chord (m)

%% --- NONLINEAR STALL AND DRAG PARAMETERS ---
P.M = 50; % Stall blending function slope
P.alpha0 = 0.47; % Stall angle of attack (rad)
P.e = 0.9; % Oswald efficiency factor

%% --- LONGITUDINAL AERODYNAMIC COEFFICIENTS ---
% Subscripts:
% 0 = zero angle of attack
% alpha = angle of attack derivative
% q = pitch-rate derivative
% de = elevator deflection derivative

% Lift
P.C_L0 = 0.23;
P.C_Lalpha = 5.61;
P.C_Lq = 7.95;
P.C_Lde = 0.13;

% Drag
P.C_D0 = 0.0424;
P.C_Dalpha = 0.132;
P.C_Dq = 0;
P.C_Dde = 0.0135;
P.C_Dp = 0.043;

% Pitch moment
P.C_My0 = 0.0135;
P.C_Myalpha = -2.74;
P.C_Myq = -38.21;
P.C_Myde = -0.99;

%% --- LATERAL AERODYNAMIC COEFFICIENTS ---
% Subscripts:
% beta = sideslip angle
% p = roll rate
% r = yaw rate
% da = aileron deflection
% dr = rudder deflection

% Side force
```

```

P.C_Y0      = 0;
P.C_Ybeta   = -0.83;
P.C_Yp      = 0;
P.C_Yr      = 0;
P.C_Yda     = 0.075;
P.C_Ydr     = 0.19;

% Roll moment
P.C_Mx0     = 0;
P.C_Mxbeta  = -0.13;
P.C_Mxp     = -0.51;
P.C_Mxr     = 0.25;
P.C_Mxda    = 0.17;
P.C_Mxdr    = 0.0024;

% Yaw moment
P.C_Mz0     = 0;
P.C_Mzbeta  = 0.073;
P.C_Mzp     = -0.069;
P.C_Mzr     = -0.095;
P.C_Mzda    = -0.011;
P.C_Mzdr    = -0.069;

%% --- PROPULSION PARAMETERS (MOTOR & BATTERY) ---
P.V_max     = 44.4;      % Maximum battery voltage (V)
P.k_V       = 0.0659;    % Motor constant (V·s/rad)
P.k_Q       = 0.0659;    % Motor torque constant (N·m/A)
P.R_motor   = 0.042;     % Motor winding resistance (Ohms)
P.i0        = 1.5;       % No-load current (A)

%% --- PROPELLER AERODYNAMIC COEFFICIENTS ---
% Torque coefficient polynomial
P.C_Q0 = 0.005230;
P.C_Q1 = 0.004970;
P.C_Q2 = -0.01664;

% Thrust coefficient polynomial
P.C_T0 = 0.09357;
P.C_T1 = -0.06044;
P.C_T2 = -0.1079;

P.D_prop = 0.508;      % Propeller diameter (m)

%% --- ATMOSPHERIC PARAMETERS ---
P.T0 = 288.15;         % Standard temperature at sea level (K)
P.p0 = 101325;         % Standard pressure at sea level (Pa)
P.L0 = -0.0065;        % Temperature lapse rate (K/m)
P.R = 8.31432;         % Universal gas constant (J/(mol·K))
P.Molar = 0.0289644;   % Molar mass of air (kg/mol)
P.rho = 1.268;         % Air density (kg/m^3)

```

compute_tf_coeffs.m

```

function TF = compute_tf_coeffs(Va, alpha, theta, de, dt, P)
% computes longitudinal and lateral transfer function coefficients
% inputs:
% Va: [1x1] trim airspeed (m/s)
% alpha: [1x1] trim angle of attack (rad)
% theta: [1x1] trim pitch angle (rad)
% de: [1x1] trim elevator (rad)
% dt: [1x1] trim throttle (unitless)
% P: [1x1 struct] physical parameters
% outputs:
% TF: [1x1 struct] transfer function coefficients

% 1. dynamic pressure and air density (assuming sea level for trim)
rho = P.rho;

% 2. lateral coefficients (roll subsystem)
cp_p = P.Gamma3 * P.C_Mxp + P.Gamma4 * P.C_Mzp;
cp_da = P.Gamma3 * P.C_Mxda + P.Gamma4 * P.C_Mzda;

TF.a_phi1 = -0.5 * rho * Va^2 * P.S * P.b * cp_p * (P.b / (2 * Va));
TF.a_phi2 = 0.5 * rho * Va^2 * P.S * P.b * cp_da;

```

```

% 3. longitudinal coefficients (pitch subsystem)
TF.a_theta1 = -(rho * Va^2 * P.c * P.S / (2 * P.Jy)) * P.C_Myq * (P.c / (2 * Va));
TF.a_theta2 = -(rho * Va^2 * P.c * P.S / (2 * P.Jy)) * P.C_Myalpha;
TF.a_theta3 = (rho * Va^2 * P.c * P.S / (2 * P.Jy)) * P.C_Myde;

% 4. longitudinal coefficients (airspeed subsystem)
eps_num = 1e-4;

% numerical derivative of thrust w.r.t airspeed (dTp_dVa)
[f_p_plus, ~] = get_propulsion(Va + eps_num, dt, rho, P);
[f_p_minus, ~] = get_propulsion(Va - eps_num, dt, rho, P);
dtp_dva = (f_p_plus(1) - f_p_minus(1)) / (2 * eps_num);

% numerical derivative of thrust w.r.t throttle (dTp_ddt)
[f_p_plus, ~] = get_propulsion(Va, dt + eps_num, rho, P);
[f_p_minus, ~] = get_propulsion(Va, dt - eps_num, rho, P);
dtp_ddt = (f_p_plus(1) - f_p_minus(1)) / (2 * eps_num);

% linear drag coefficient derivative
ar = (P.b^2) / P.S;
cd_alpha = 2 * (P.C_L0 + P.C_Lalpha * alpha) * P.C_Lalpha / (pi * P.e * ar);

TF.a_V1 = (rho * Va * P.S / P.mass) * (P.C_D0 + cd_alpha * alpha + P.C_Dde * de) - (1 / P.mass) * dtp_dva;
TF.a_V2 = (1 / P.mass) * dtp_ddt;
TF.a_V3 = P.g * cos(theta - alpha);
end

function [F_p, M_p] = get_propulsion(Va, dt, rho, P)
% localized propulsion logic for numerical derivative calculations
if dt == 0
    F_p = [0;0;0]; M_p = [0;0;0]; return;
end
v_in = P.V_max * dt;
a = (rho * P.D_prop^5) / ((2 * pi)^2) * P.C_Q0;
b = (rho * P.D_prop^4) / (2 * pi) * P.C_Q1 * Va + (P.k_Q * P.k_V) / P.R_motor;
c = rho * P.D_prop^3 * P.C_Q2 * Va^2 - (P.k_Q / P.R_motor) * v_in + P.k_Q * P.i0;
omega_p = (-b + sqrt(b^2 - 4 * a * c)) / (2 * a);

t_p = (rho * P.D_prop^4) / ((2 * pi)^2) * omega_p^2 * P.C_T0 + (rho * P.D_prop^3) / (2 * pi) * omega_p * Va * P.C_T1 + rho * P.D_prop^2 * Va^2 * P.C_T2;
q_p = (rho * P.D_prop^5) / ((2 * pi)^2) * omega_p^2 * P.C_Q0 + (rho * P.D_prop^4) / (2 * pi) * omega_p * Va * P.C_Q1 + rho * P.D_prop^3 * Va^2 * P.C_Q2;
F_p = [t_p; 0; 0]; M_p = [-q_p; 0; 0];
end

```

update_uav_plot.m

```

function update_uav_plot(pn_pe_pd, phi_theta_psi, path_cmd)
% persistent variables keep the figure open and update it every step
persistent fig_handle patches traj_actual ...
    traj_desired p_history last_update_time

% fps limiter (caps graphics to ~25 FPS to prevent lag)
if isempty(last_update_time)
    last_update_time = tic;
elseif toc(last_update_time) < 0.04
    return;
else
    last_update_time = tic;
end

% extract states
pn = pn_pe_pd(1);
pe = pn_pe_pd(2);
alt = -pn_pe_pd(3);

phi = phi_theta_psi(1);
theta = phi_theta_psi(2);
psi = phi_theta_psi(3);

% initialize figure
if isempty(fig_handle) || ~isvalid(fig_handle)
    fig_handle = figure('Name', '3D UAV Visualization', 'Color', ...
        'w', 'NumberTitle', 'off');
    ax = axes('Parent', fig_handle, 'Color', [0.98 0.98 0.98]);
end

```

```

hold(ax, 'on'); grid(ax, 'on'); view(ax, 3);

% scale to fit the figure-8 and helix
axis(ax, [100 900 100 900 -100 500]);
xlabel(ax, 'East (m)', 'FontWeight', 'bold');
ylabel(ax, 'North (m)', 'FontWeight', 'bold');
zlabel(ax, 'Altitude (m)', 'FontWeight', 'bold');

% generate and plot the full desired path
if length(path_cmd) >= 4
    traj_type = path_cmd(4);
    A = 300;
    N_center = 500;
    E_center = 500;

    if traj_type == 1 % figure-8
        theta_full = linspace(0, 2*pi, 200); % full geometric loop
        N_full = N_center + A * sin(theta_full) .* cos(theta_full);
        E_full = E_center + A * sin(theta_full);
        Alt_full = 100 * ones(size(theta_full));
    else % helix
        theta_full = linspace(0, 8*pi, 400); % 4 spiral loops
        N_full = N_center + A * cos(theta_full);
        E_full = E_center + A * sin(theta_full);
        Alt_full = 100 + (13.0 * theta_full);
    end

    % draw the full static path (red dashed line)
    plot3(ax, E_full, N_full, Alt_full, 'r--', 'LineWidth', ...
        1.5, 'DisplayName', 'Desired Path');

    % initialize the moving target "carrot" (red dot)
    traj_desired = plot3(ax, path_cmd(2), path_cmd(1), path_cmd(3), ...
        'ro', 'MarkerSize', 6, ...
        'MarkerFaceColor', 'r', ...
        'DisplayName', 'Target');

end

% plot actual path (blue)
traj_actual = plot3(ax, pe, pn, alt, 'b-', 'LineWidth', 1.5, ...
    'DisplayName', 'Actual Path');
p_history = [pe; pn; alt];

% enforce only the trajectory lines in the legend
legend(ax, 'Location', 'northeast');

% visualize the uav
S = 3.0; % size scale
uav_color = [0.6 0.6 0.6]; % grey color

% vertices (body frame: x-forward, y-right, z-down)
V_fuse = [1 0 0;
    0 0.1 0.1;
    0 -0.1 0.1;
    0 0.1 -0.1;
    0 -0.1 -0.1;
    -3 0 0] * S;
F_fuse = [1 2 4;
    1 4 3;
    1 3 5;
    1 5 2;
    6 2 4;
    6 4 3;
    6 3 5;
    6 5 2];

V_wing = [ 0 2 0;
    -0.5 2 0;
    -0.5 -2 0;
    0 -2 0] * S;
F_wing = [1 2 3 4];

V_htail = [-2.5 0.7 0;
    -3 0.7 0;
    -3 -0.7 0;
    -2.5 -0.7 0] * S;
F_htail = [1 2 3 4];

```

```

V_vtail = [-2.5 0 0;
           -3 0 0;
           -3 0 -1;
           -2.5 0 -1] * S;
F_vtail = [1 2 3 4];

% save geometry
patches.v_fuse = V_fuse; patches.v_wing = V_wing;
patches.v_htail = V_htail; patches.v_vtail = V_vtail;

% render (HandleVisibility = 'off' keeps them out of the legend)
patches.h_fuse = patch(ax, 'Vertices', V_fuse, 'Faces', F_fuse, ...
    'FaceColor', uav_color, 'EdgeColor', 'k', ...
    'HandleVisibility', 'off');
patches.h_wing = patch(ax, 'Vertices', V_wing, 'Faces', F_wing, ...
    'FaceColor', uav_color, 'EdgeColor', 'k', ...
    'HandleVisibility', 'off');
patches.h_htail = patch(ax, 'Vertices', V_htail, 'Faces', F_htail, ...
    'FaceColor', uav_color, 'EdgeColor', 'k', ...
    'HandleVisibility', 'off');
patches.h_vtail = patch(ax, 'Vertices', V_vtail, 'Faces', F_vtail, ...
    'FaceColor', uav_color, 'EdgeColor', 'k', ...
    'HandleVisibility', 'off');

else
    % update actual trajectory path line
    p_history(:, end+1) = [pe; pn; alt];
    set(traj_actual, 'XData', p_history(1,:), ...
        'YData', p_history(2,:), ...
        'ZData', p_history(3,:));

    % update target carrot position
    if length(path_cmd) >= 3
        set(traj_desired, 'XData', path_cmd(2), ...
            'YData', path_cmd(1), ...
            'ZData', path_cmd(3));
    end
end

% update uav 3d orientation
R_roll = [1 0 0; 0 cos(phi) -sin(phi); 0 sin(phi) cos(phi)];
R_pitch = [cos(theta) 0 sin(theta); 0 1 0; -sin(theta) 0 cos(theta)];
R_yaw = [cos(psi) -sin(psi) 0; sin(psi) cos(psi) 0; 0 0 1];
R_body2ned = R_yaw * R_pitch * R_roll;

% map NED body coordinates to plot axes (x=east, y=north, z=up)
C_ned2plot = [0 1 0; 1 0 0; 0 0 -1];
R_total = C_ned2plot * R_body2ned;

tf = @(v) (R_total * v)' + [pe, pn, alt];

% apply math to graphics objects
set(patches.h_fuse, 'Vertices', tf(patches.v_fuse));
set(patches.h_wing, 'Vertices', tf(patches.v_wing));
set(patches.h_htail, 'Vertices', tf(patches.v_htail));
set(patches.h_vtail, 'Vertices', tf(patches.v_vtail));

drawnow limitrate;
end

```

STM32

1. Configuration

1.1. Project Manager

STM32CubeMX Untitled - STM32F407VGtx

File Window Help

Home > STM32F407VGtx > Untitled - Project Manager > GENERATE CODE

Pinout & Configuration	Clock Configuration	Project Manager	Tools
Project	Project Settings Project Name: uav_fixedwing Project Location: C:\Users\ADMIN\Documents\Project\quadcopter-simulink\firmware Browse Application Structure: Advanced ☐ Do not generate the main() Toolchain Folder Location: C:\Users\ADMIN\Documents\Project\quadcopter-simulink\firmware\uav_fixedwing\ Toolchain / IDE: CMake ☐ Generate Under Root Default Compiler/Linker: GCC		
Code Generator	Linker Settings Minimum Heap Size: 0x200 Minimum Stack Size: 0x400		
Advanced Settings	Thread-safe Settings Cortex-MNVS ☐ Enable multi-threaded support Thread-safe Locking Strategy: Default - Mapping suitable strategy depending on RTOS selection		
	Mcu and Firmware Package Mcu Reference: STM32F407VGtx Firmware Package Name and Version: STM32Cube_FW_F4_V1.28.3 ☒ Use latest available version ☒ Use Default Firmware Location Firmware Relative Path: C:\Users\ADMIN\STM32Cube\Repository\STM32Cube_FW_F4_V1.28.3 Browse		

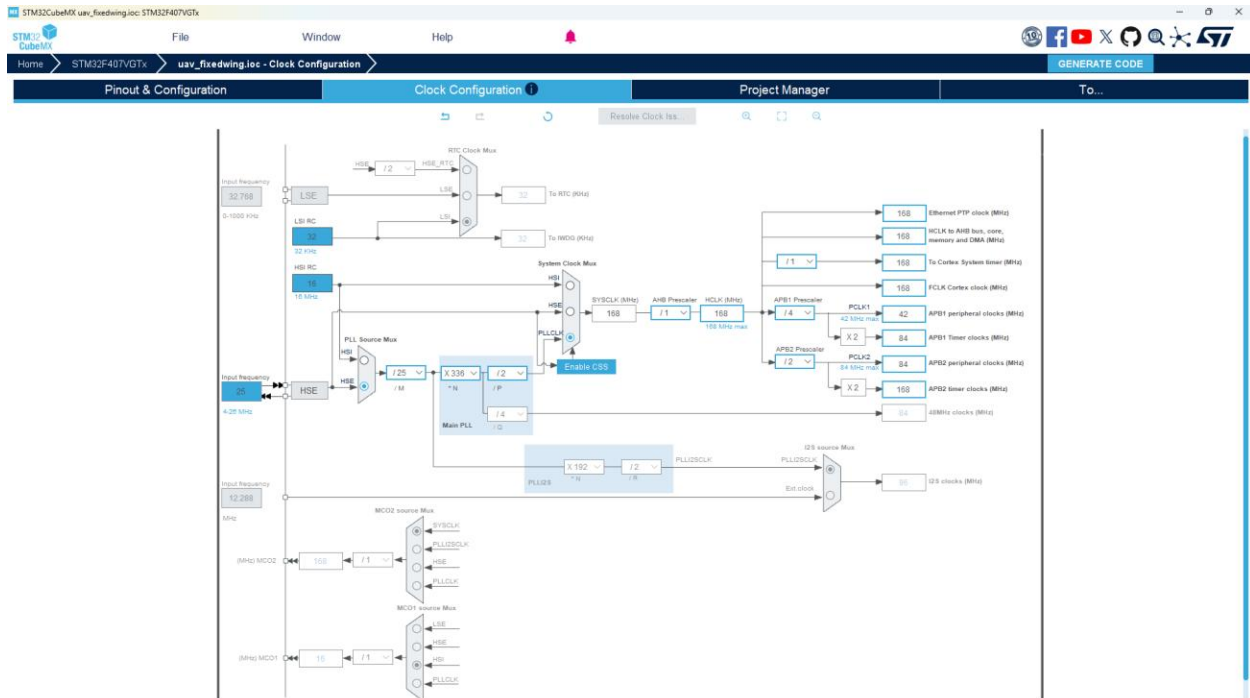
STM32CubeMX Untitled - STM32F407VGtx

File Window Help

Home > STM32F407VGtx > Untitled - Project Manager > GENERATE CODE

Pinout & Configuration	Clock Configuration	Project Manager	Tools
Project	STM32Cube MCU packages and embedded software packs ☐ Copy all used libraries into the project folder ☒ Copy only the necessary library files ☐ Add necessary library files as reference in the toolchain project configuration file		
Code Generator	Generated files ☒ Generate peripheral initialization as a pair of *.c/.h files per peripheral ☐ Backup previously generated files when re-generating ☒ Keep User Code when re-generating ☒ Delete previously generated files when not re-generated		
Advanced Settings	HAL Settings ☐ Set all free pins as analog (to optimize the power consumption) ☐ Enable Full Assert		
	User Actions Before Code Generation: Browse After Code Generation: Browse		
	Template Settings Select a template to generate customized code. Settings		

1.2. Clock Configuration



1.3. Pinout & Configuration

SYS Mode and Configuration

Mode

Debug

☐ System Wake-Up

Timebase Source

STM32CubeMX uav_fixwing.ios: STM32F407VGTx

File Window Help

Home > STM32F407VGTx > uav_fixwing.ios - Pinout & Configuration

Pinout & Configuration Clock Configuration Project Manager To...

Categories: A-Z

System Core >

Analog >

Timers >

RTC

TIM1

TIM3

TIM4

TIM5

TIM6

TIM7

TIM8

TIM9

TIM10

TIM11

TIM12

TIM13

TIM14

Connectivity >

Multimedia >

Security >

Computing >

Middleware and Software Packs >

Mode

Slave Mode ☐ Disable

Trigger Source ☐ Disable

Clock Source ☐ Internal Clock

Channel1 ☐ Disable

Channel2 ☐ Disable

Channel3 ☐ Disable

Channel4 ☐ Disable

Combined Channels ☐ Disable

☐ Use ETR as Clearing Source

☐ XOR activation

☐ One Pulse Mode

Configuration

Reset Configuration

Parameter Settings User Constants NVIC Settings DMA Settings

Configure the below parameters

Counter Settings

Prescaler (PSC - 16 bits value) 8400-1

Counter Mode Up

Counter Period (AutoReload Register - 16 bits value) 100-1

Internal Clock Division (CKD) No Division

auto-reload preload Disable

Trigger Output (TRGO) Parameters

Master/Slave Mode (MSM bit) Disable (Trigger input effect not delayed)

Trigger Event Selection Reset (JG bit from TIMx_EGR)

Pinout view System view

STM32F407VGTx LQFP100

Configuration

Reset Configuration

Parameter Settings User Constants NVIC Settings DMA Settings GPIO Settings

DMA Request	Stream	Direction	Priority
USART2_RX	DMA1 Stream 5	Peripheral To Memory	Low
USART2_TX	DMA1 Stream 6	Memory To Peripheral	Low

Add Delete

DMA Request Settings

Peripheral		Memory	
Mode	Circular	Increment Address	☒
Use Fifo	☐	Data Width	Byte
Threshold		Burst Size	

Configuration

Reset Configuration

Parameter Settings

User Constants

NVIC Settings

DMA Settings

GPIO Settings

DMA Request	Stream	Direction	Priority
USART2_RX	DMA1 Stream 5	Peripheral To Memory	Low
USART2_TX	DMA1 Stream 6	Memory To Peripheral	Low

Add

Delete

DMA Request Settings

Mode

Normal

Increment Address☐

Peripheral

Memory☒

Use Fifo☐

Threshold

Data Width

Byte

Burst Size

Byte

Configuration

Reset Configuration

Parameter Settings

User Constants

NVIC Settings

DMA Settings

GPIO Settings

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
DMA1 stream5 global interrupt	☒	5	0
DMA1 stream6 global interrupt	☒	5	0
USART2 global interrupt	☒	5	0

Configuration

Reset Configuration

Tasks and Queues

Timers and Semaphores

Mutexes

Events

FreeRTOS Heap Usage

Config parameters

Include parameters

Advanced settings

User Constants

Tasks

Task Name	Priority	Stack Size (...)	Entry Function	Code Generati...	Parameter	Allocation	Buffer Name	Control Block ...
autopilot_task	osPriorityHigh	1024	start_autopilot...	Default	NULL	Dynamic	NULL	NULL
rx_task	osPriorityNor...	1024	start_rx_task	Default	NULL	Dynamic	NULL	NULL

Add

Delete

Queues

Queue Name	Queue Size	Item Size	Allocation	Buffer Name	Control Block Name
------------	------------	-----------	------------	-------------	--------------------

Add

Delete

2. Programming